Ex No : 8 Date : 08/10/26	MODEL EVALUATION AND IMPROVEMENT: HYPERPARAMETER TUNING WITH GRID SEARCH AND CROSS-VALIDATION

AIM:

To demonstrate model evaluation and improvement techniques using **Grid Search and K-Fold CrossValidation**. Grid Search is used to find the best hyperparameters for an SVM classifier, while K-Fold Cross-Validation is used to obtain a reliable estimate of model performance.

ALGORITHM:

Part 1: Hyperparameter Tuning using Grid Search

1. Import the required Python libraries and load the Iris dataset.
2. Separate the dataset into features (X) and target labels (y).
3. Split the dataset into training and testing sets using train_test_split().
4. Standardize the training and testing features using StandardScaler.
5. Create an SVM classifier using SVC.
6. Define a parameter grid containing different values of:
   - C
   - gamma
   - kernel
7. Apply GridSearchCV with 5-fold cross-validation to test all possible combinations.
8. Select the hyperparameter combination with the highest cross-validation accuracy.
9. Use the best model to predict the test data.
10. Calculate the test accuracy, classification report, and confusion matrix.
11. Display the top Grid Search results.

Part 2: K-Fold Cross-Validation

1. Create an SVM model with default parameters.
2. Define a 5-fold K-Fold Cross-Validation strategy.
3. Divide the training data into 5 folds.
4. Train the model using 4 folds and validate it using the remaining fold.
5. Repeat the process 5 times so that every fold is used once for validation.
6. Calculate the accuracy for each fold.
7. Calculate the mean and standard deviation of the accuracy scores.
8. Visualize the cross-validation scores using a bar graph.

CODE:

```
# Import necessary libraries
import numpy as np    Import "numpy" could not be resolved
import pandas as pd    Import "pandas" could not be resolved from
import matplotlib.pyplot as plt    Import "matplotlib.pyplot" cou
import seaborn as sns    Import "seaborn" could not be resolved f

from sklearn.datasets import load_iris    Import "sklearn.dataset
from sklearn.model_selection import (    Import "sklearn.model_se
    train_test_split,
    KFold,
    cross_val_score,
    GridSearchCV
)
from sklearn.svm import SVC    Import "sklearn.svm" could not be
from sklearn.metrics import (    Import "sklearn.metrics" could n
    accuracy_score,
    classification_report,
    confusion_matrix
)
from sklearn.preprocessing import StandardScaler    Import "sklea


# ================================================
# PART 1: HYPERPARAMETER TUNING WITH GRID SEARCH
# ================================================

print("--- Part 1: Hyperparameter Tuning with Grid Search ---")

# 1. Load Iris Dataset
iris = load_iris()

X = iris.data
y = iris.target
```

```
feature_names = iris.feature_names
target_names = iris.target_names

print("\nDataset Features (X) shape:", X.shape)
print("Dataset Labels (y) shape:", y.shape)
print("Feature Names:", feature_names)
print("Target Names:", target_names)


# 2. Split Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

print("\nTraining set size:", X_train.shape[0])
print("Test set size:", X_test.shape[0])


# 3. Standardize Features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeatures standardized.")


# 4. Define SVM Model and Hyperparameter Grid
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear']
```

```
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear']
}

print("\nHyperparameter grid defined:")

for param, values in param_grid.items():
    print(param, ":", values)


# 5. Perform Grid Search with 5-Fold Cross-Validation
grid_search = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,
    scoring='accuracy',
    verbose=1,
    n_jobs=-1
)

print("\nStarting Grid Search...")

grid_search.fit(X_train_scaled, y_train)

print("\nGrid Search completed.")


# 6. Display Best Parameters and Score
print("\nBest hyperparameters found:")
print(grid_search.best_params_)

print(
    "Best cross-validation accuracy:",
    f"{grid_search.best_score_:.4f}"
)
```

```python
# 7. Evaluate Best Model on Test Data
best_model = grid_search.best_estimator_

y_pred_tuned = best_model.predict(X_test_scaled)

test_accuracy_tuned = accuracy_score(
    y_test,
    y_pred_tuned
)

print(
    "\nTest set accuracy with tuned model:",
    f"{test_accuracy_tuned:.4f}"
)


# 8. Classification Report
print("\n--- Classification Report for Tuned Model ---")

print(
    classification_report(
        y_test,
        y_pred_tuned,
        target_names=target_names
    )
)


# 9. Confusion Matrix
print("\n--- Confusion Matrix for Tuned Model ---")

cm_tuned = confusion_matrix(
    y_test,
    y_pred_tuned
)
```

```
print("\n--- Confusion Matrix for Tuned Model ---")

cm_tuned = confusion_matrix(
    y_test,
    y_pred_tuned
)

plt.figure(figsize=(8, 6))

sns.heatmap(
    cm_tuned,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=target_names,
    yticklabels=target_names
)

plt.title('Confusion Matrix (Tuned SVM)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()


# 10. Display Top Grid Search Results
results_df = pd.DataFrame(
    grid_search.cv_results_
)

print("\n--- Top 5 Grid Search Results ---")

print(
    results_df[
        [
            'param_C',
            'param_gamma',
```

```
            'param_C',
            'param_gamma',
            'param_kernel',
            'mean_test_score',
            'rank_test_score'
        ]
    ]
    .sort_values(by='rank_test_score')
    .head()
)


# ================================================
# PART 2: K-FOLD CROSS-VALIDATION
# ================================================

print("\n--- Part 2: K-Fold Cross-Validation ---")


# 1. Create SVM Model
model_cv = SVC(random_state=42)


# 2. Define 5-Fold Cross-Validation
k_folds = 5

kf = KFold(
    n_splits=k_folds,
    shuffle=True,
    random_state=42
)

print(
    f"\nPerforming {k_folds}-fold cross-validation..."
)
```

```
)


# 3. Calculate Cross-Validation Scores
cv_scores = cross_val_score(
    model_cv,
    X_train_scaled,
    y_train,
    cv=kf,
    scoring='accuracy'
)


# 4. Display Results
print("\nCross-validation scores for each fold:")
print(cv_scores)

print(
    "Mean cross-validation accuracy:",
    f"{np.mean(cv_scores):.4f}"
)

print(
    "Standard deviation of cross-validation accuracy:",
    f"{np.std(cv_scores):.4f}"
)


# 5. Visualize Cross-Validation Scores
plt.figure(figsize=(8, 5))

plt.bar(
    range(1, k_folds + 1),
    cv_scores
)
```

```
plt.axhline(
    y=np.mean(cv_scores),
    linestyle='--',
    label=f'Mean Accuracy = {np.mean(cv_scores):.4f}'
)

plt.xlabel('Fold Number')
plt.ylabel('Accuracy')
plt.title('5-Fold Cross-Validation Accuracy')
plt.xticks(range(1, k_folds + 1))
plt.legend()

plt.show()
```

OUTPUT:

```
--- Part 1: Hyperparameter Tuning with Grid Search ---

Dataset Features (X) shape: (150, 4)
Dataset Labels (y) shape: (150,)
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target Names: ['setosa' 'versicolor' 'virginica']

Training set size: 105
Test set size: 45

Features standardized.

Hyperparameter grid defined:
C : [0.1, 1, 10, 100]
gamma : [1, 0.1, 0.01, 0.001]
kernel : ['rbf', 'linear']

Starting Grid Search...
Fitting 5 folds for each of 32 candidates, totalling 160 fits

Grid Search completed.

Best hyperparameters found:
{'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
Best cross-validation accuracy: 0.9810

Test set accuracy with tuned model: 0.9111

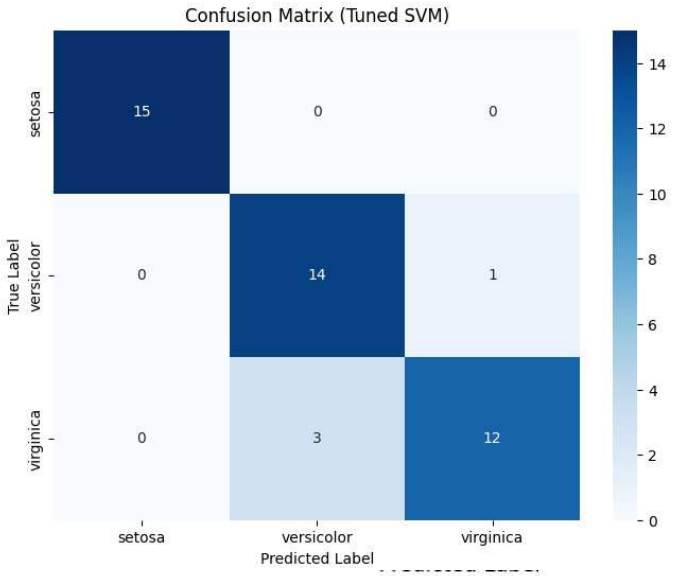
--- Classification Report for Tuned Model ---
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.82      0.93      0.88        15
   virginica       0.92      0.80      0.86        15

    accuracy                           0.91        45
   macro avg       0.92      0.91      0.91        45
weighted avg       0.92      0.91      0.91        45


--- Confusion Matrix for Tuned Model ---
```

```
--- Confusion Matrix for Tuned Model ---
```

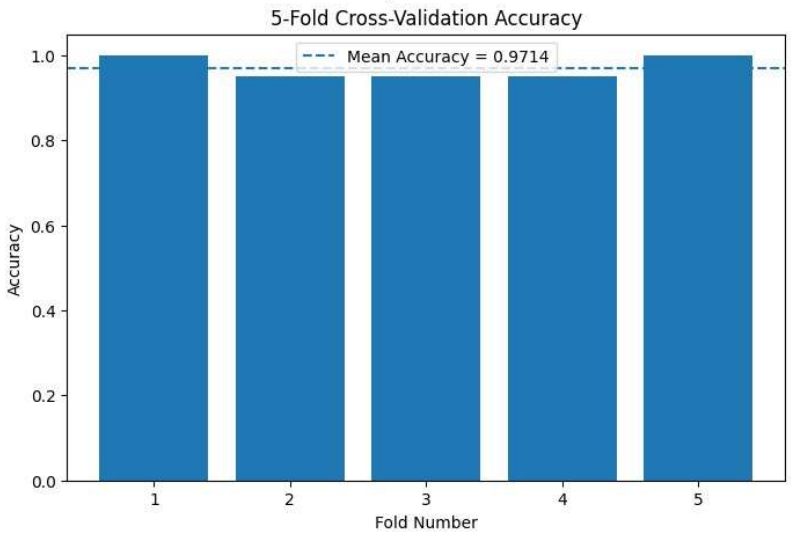



```
--- Top 5 Grid Search Results ---
    param_C  param_gamma param_kernel  mean_test_score  rank_test_score
10      1.0        0.100          rbf         0.980952                1
27    100.0        0.100       linear         0.980952                1
31    100.0        0.001       linear         0.980952                1
29    100.0        0.010       linear         0.980952                1
25    100.0        1.000       linear         0.980952                1

--- Part 2: K-Fold Cross-Validation ---

Performing 5-fold cross-validation...

Cross-validation scores for each fold:
[1.         0.95238095 0.95238095 0.95238095 1.        ]
Mean cross-validation accuracy: 0.9714
Standard deviation of cross-validation accuracy: 0.0233
```



RESULT:

The Iris dataset was successfully evaluated using **SVM classification**. Grid Search with 5-fold cross-validation was used to identify the best combination of C, gamma, and kernel parameters. The tuned SVM model was then evaluated on the test dataset using accuracy, classification report, and confusion matrix.
K-Fold Cross-Validation was also successfully implemented using **5 folds**. The accuracy obtained from each fold, along with the **mean and standard deviation**, was calculated to provide a more reliable estimate of model performance.